UPSSA
UNIVERSITÉ PRIVÉE DE LA SANTÉ ET DES SCIENCES D'AGADIR
الجامعة الخاصة للصحة و العلوم لأكادير
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⵉⵖⵓⵔⵉⵜ ⵏ ⵜⵎⴰⵡⴰⵙ ⴷ ⵜⵜⵓⵖⵓⵍⵉⵏ ⵏ ⵗⴳⴰⴷⵉⵔ

ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE D'AGADIR
المدرسة العليا الخاصة للهندسة بأكادير
ⵜⵉⵎⵙⵙⵓⵔⵉⵏ ⵜⵜⵓⵖⵓⵍⵉⵏⵜ ⵜⵉⵖⵓⵔⵉⵜ ⵏ ⵗⴳⴰⴷⵉⵔ

TRAVAUX PRATIQUES

Bases de Données & Modélisation des SI

Atelier 4 : Filtrer les données avec WHERE

Objectif : sélectionner les lignes répondant à une condition, comparer des valeurs ou des colonnes, filtrer sur une date, utiliser LEFT et distinguer les valeurs renseignées des valeurs NULL.

Prérequis : SELECT, FROM, choix des colonnes, commentaires et Query Tool. Base de travail : AdventureWorks sous PostgreSQL.

Le support alterne notions de cours, étapes guidées et observations. Les conditions uniques constituent le cœur du TP ; AND, OR et NOT sont seulement présentés en ouverture. Un travail à rendre figure à la fin.

I- Préparer les tables et le fichier de travail

1. Connectez-vous à AdventureWorks dans pgAdmin, puis ouvrez Query Tool sur cette base. Enregistrez un nouveau fichier TP4_Nom_Prenom.sql.

2. Dans l'explorateur, repérez production.product et ses colonnes productid, name, productnumber, color, listprice, standardcost, safetystocklevel, reorderpoint, sellstartdate et sellenddate. Repérez également person.person avec firstname et lastname.

Les requêtes emploient les noms non délimités en minuscules, habituels dans certaines conversions PostgreSQL. Si votre archive utilise des noms avec majuscules, adaptez-les exactement entre guillemets doubles, par exemple "Production"."Product" et "ListPrice". Ne créez pas une colonne parce que son nom diffère : consultez d'abord la structure.

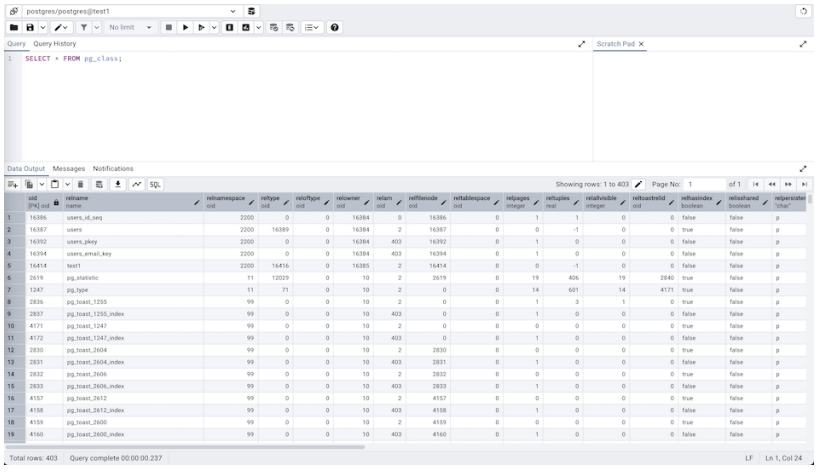


Figure 1 — Éditeur SQL et résultat dans pgAdmin, capture officielle de démonstration

II- Comprendre le rôle de WHERE

SELECT détermine les colonnes affichées. WHERE détermine les lignes conservées. Pour chaque ligne de la source, le SGBD évalue une condition appelée prédicat. Seules les lignes pour lesquelles cette condition vaut TRUE sont retenues.

```
SELECT productid, name, color
FROM production.product
WHERE color = 'Black';
```

Lecture : afficher l'identifiant, le nom et la couleur des produits dont la couleur est Black. FROM précise la table ; WHERE vient après FROM. Le point-virgule est placé à la fin de la condition, et non avant WHERE.

Une comparaison ordinaire produit TRUE ou FALSE quand ses valeurs sont renseignées. Si une valeur est NULL, elle peut produire UNKNOWN, représenté par NULL. WHERE écarte à la fois FALSE et UNKNOWN : cette précision sera utilisée dans la partie sur les valeurs absentes.

3. Exécutez d'abord

```
SELECT productid, name, color
FROM production.product
```

puis la version filtrée ci-dessus. Comparez les colonnes et les lignes.

Résultat attendu : Les colonnes demandées sont inchangées ; les lignes retournées correspondent au critère de couleur. Un résultat vide est possible si aucune valeur ne satisfait la condition dans votre archive.

III- Écrire une égalité sur un texte ou un nombre

L'égalité s'écrit avec un seul signe =. Un texte s'écrit entre apostrophes simples : 'Black'. Les guillemets doubles désignent des identifiants PostgreSQL, pas des chaînes de caractères. Un nombre s'écrit sans apostrophes et utilise le point comme séparateur décimal.

4. Exécutez le filtre sur le texte. Vérifiez que la valeur exacte existe dans color ; adaptez-la si nécessaire.

```
SELECT productid, name, color
FROM production.product
WHERE color = 'Black';
```

5. Choisissez l'identifiant d'un produit réellement présent, puis remplacez la valeur 1 de l'exemple et exécutez la requête.

```
SELECT productid, name, productnumber
FROM production.product
WHERE productid = 1;
```

Le 1 est une valeur de comparaison, pas une colonne. La valeur '1' est un texte : PostgreSQL peut effectuer certaines conversions implicites, mais écrivez un nombre comme un nombre pour exprimer clairement votre intention.

La comparaison des textes dépend des règles du type et de la collation. Avec les types text/varchar et les collations usuelles, Black et black sont différents. Reprenez la casse des données au lieu de supposer que les mots-clés et les valeurs suivent la même règle.

IV- Filtrer sur une colonne sans l'afficher

6. Exécutez la requête suivante, qui utilise color dans la condition sans l'inclure dans SELECT.

```
SELECT productnumber, name
FROM production.product
WHERE color = 'Black';
```

Le serveur dispose des colonnes de la table pour évaluer WHERE, même si elles n'apparaissent pas dans le résultat. La sélection des colonnes et le critère de filtrage ont deux rôles différents.

À faire : Ajoutez temporairement color dans SELECT pour contrôler le filtre, puis retirez-la. Les produits sélectionnés doivent rester les mêmes, même si leur ordre d'affichage peut varier sans ORDER BY.

Résultat attendu : Le résultat final comporte exactement deux colonnes. Toutes les lignes ont été retenues d'après color, bien que cette couleur ne soit plus visible.

V- Maîtriser les opérateurs de comparaison

Les opérateurs ci-dessous comparent la valeur de gauche à celle de droite. Pour <= et >=, le chevron est placé avant le signe égal. PostgreSQL accepte <> et != pour « différent de » ; <> est l'écriture SQL standard.

Opérateur	Signification	Exemple vrai
=	égal à	10 = 10
<>	différent de	10 <> 20
<	strictement inférieur à	10 < 20
>	strictement supérieur à	20 > 10
<=	inférieur ou égal à	10 <= 10
>=	supérieur ou égal à	10 >= 10

7. Affichez les produits dont le prix catalogue est strictement supérieur à 1000.

```
SELECT productnumber, name, listprice
FROM production.product
WHERE listprice > 1000;
```

8. Remplacez > par >=, puis par < et <= dans des essais séparés. Avant chaque exécution, écrivez en commentaire le sens de la condition.

Pour observer précisément la frontière, choisissez un prix existant dans la table, puis comparez > et >= avec ce même prix. Les produits dont le prix égale la frontière sont exclus par > et inclus par >=.

9. Affichez les produits dont la couleur renseignée est différente de Black.

```
SELECT productnumber, name, color
FROM production.product
WHERE color <> 'Black';
```

Cette dernière condition ne sélectionne pas les couleurs NULL. Une information absente n'est pas automatiquement une valeur différente de Black : son test de comparaison ordinaire reste inconnu.

VI- Filtrer sur une date

Une date ne s'écrit pas comme un nombre non délimité. Utilisez un littéral typé au format ISO : DATE '2013-01-01'. Sans apostrophes, 2013-01-01 est interprétable comme un calcul arithmétique, pas comme cette date.

10. Affichez les produits dont la date de début de commercialisation est au moins le 1er janvier 2013.

```
SELECT productnumber, name, sellstartdate
FROM production.product
WHERE sellstartdate >= DATE '2013-01-01';
```

Vérifiez le type de sellstartdate : selon la conversion, il peut être date ou timestamp. Un timestamp inclut une heure. Une comparaison avec DATE utilise la frontière du début de journée ; pour les types avec fuseau horaire, le contexte de fuseau doit également être pris en compte.

11. Pour les produits commercialisés avant le 1er janvier 2014, utilisez la condition ci-dessous.

```
SELECT productnumber, name, sellstartdate
FROM production.product
WHERE sellstartdate < DATE '2014-01-01';
```

La frontière < DATE '2014-01-01' couvre les dates antérieures, y compris les heures du 31 décembre 2013 pour un timestamp. Pour filtrer une seule journée avec des heures, une égalité à minuit ne suffit généralement pas : un intervalle sera étudié avec AND..

VII- Comparer deux colonnes d'une même ligne

La partie droite d'une comparaison peut être une autre colonne compatible, au lieu d'une constante. Le serveur compare alors les deux valeurs de chaque ligne, sans comparer toutes les lignes entre elles.

12. Comparez le prix catalogue et le coût standard de chaque produit.

```
SELECT productnumber, name, listprice, standardcost
FROM production.product
WHERE listprice > standardcost;
```

La ligne est retenue si son propre listprice dépasse son propre standardcost. Cette observation ne suffit pas à calculer une rentabilité complète : d'autres coûts ou prix de vente effectifs peuvent intervenir.

13. Comparez le niveau de stock de sécurité et le point de réapprovisionnement, deux colonnes de configuration du produit.

```
SELECT productnumber, name, safetystocklevel, reorderpoint
FROM production.product
WHERE safetystocklevel > reorderpoint;
```

safetystocklevel et reorderpoint ne représentent pas ici le stock réellement disponible. Comparer un stock courant à un seuil nécessiterait les données de stock adaptées, éventuellement reliées à d'autres tables. Le présent exemple reste une comparaison de deux colonnes de la même table.

À faire : Remplacez ensuite > par = dans la requête sur les seuils et expliquez le nouveau critère. Relevez deux lignes de contrôle si elles existent ; sinon indiquez que le résultat est vide.

VIII- Utiliser une fonction dans WHERE avec LEFT

Une fonction transforme ou analyse une valeur. LEFT(texte, n) extrait les n premiers caractères en partant de la gauche. LEFT(lastname, 1) fournit donc le premier caractère du nom de famille ; LEFT(lastname, 2) en fournit les deux premiers.

14. Dans person.person, repérez lastname et firstname. Exécutez la requête suivante pour les personnes dont le nom de famille commence par J.

```
SELECT businessentityid, firstname, lastname
FROM person.person
WHERE LEFT(lastname, 1) = 'J';
```

Le filtre porte sur le résultat de la fonction, pas sur le nom complet. La fonction n'altère pas les noms stockés. Si lastname vaut NULL, LEFT(lastname, 1) vaut NULL et la comparaison ne retient pas la ligne.

15. Choisissez un préfixe de deux caractères observé dans les noms, par exemple Jo si présent, et adaptez LEFT(lastname, 2). Expliquez pourquoi ce résultat est plus restrictif que le test sur J pour ce préfixe.

IX- Comprendre et rechercher les valeurs NULL

NULL représente une valeur absente ou inconnue. Il est différent du nombre 0, d'une chaîne vide '' et du texte 'NULL'. Pour tester l'absence d'une valeur, utilisez IS NULL ; pour tester qu'une valeur est renseignée, utilisez IS NOT NULL.

16. Affichez les produits dont la couleur est absente, puis ceux dont la couleur est renseignée.

```
SELECT productnumber, name, color
FROM production.product
WHERE color IS NULL;
```

```
SELECT productnumber, name, color
FROM production.product
WHERE color IS NOT NULL;
```

Ces filtres conservent des lignes selon l'état de la colonne ; ils ne retirent pas des colonnes du résultat. IS NOT NULL inclut une chaîne vide si elle est stockée, car une chaîne vide est une valeur renseignée.

17. Testez temporairement l'écriture incorrecte ci-dessous, puis comparez-la à IS NULL et corrigez-la dans votre script final.

```
SELECT productnumber, name, color
FROM production.product
WHERE color = NULL;
```

Dans le comportement SQL habituel, color = NULL produit UNKNOWN pour chaque ligne et ne trouve pas les absences. color <> NULL ne constitue pas non plus un test de présence. Le test correct est IS NULL ou IS NOT NULL.

18. Recherchez les produits dont aucune date de fin de commercialisation n'est renseignée.

```
SELECT productnumber, name, sellenddate
FROM production.product
WHERE sellenddate IS NULL;
```

Cette requête décrit l'absence de date de fin dans cette colonne. Elle ne prouve pas à elle seule que le produit est actuellement vendu : la signification métier et les autres dates ou statuts doivent être vérifiés.

X- Vérifier les résultats et préparer les conditions composées

19. Pour chaque requête, contrôlez l'en-tête, la condition exécutée et quelques lignes. Affichez temporairement la colonne filtrée si elle n'était pas dans SELECT, puis conservez la projection demandée dans la version finale.

Une grille peut récupérer les lignes par lots. Ne confondez pas les lignes chargées à l'écran avec le total. Un résultat vide peut être correct ; vérifiez le critère et les données avant de conclure à une erreur.

Ouverture : AND impose deux conditions vraies, OR accepte au moins une condition vraie et NOT inverse une condition connue. Avec NULL, UNKNOWN doit aussi être pris en compte. Dans ce TP, ces opérateurs ne sont pas exigés dans les exercices : ils serviront aux filtres composés de la séance suivante.

Exemple à lire uniquement : WHERE listprice > 1000 AND color = 'Black' vise les produits qui vérifient les deux critères. Utilisez des parenthèses pour exprimer clairement les regroupements lorsque plusieurs opérateurs sont combinés.

XI- Erreurs fréquentes et synthèse

Point-virgule avant WHERE : il coupe l'instruction trop tôt. Texte entre guillemets doubles : le serveur cherche une colonne. Date non délimitée : elle n'est pas un littéral de date correct. Comparaison écrite => ou =< : utilisez >= ou <=. Égalité écrite == : utilisez =. Test = NULL : remplacez-le par IS NULL. Colonne inexistante : vérifiez son nom dans l'explorateur et la base connectée.

À retenir : SELECT choisit les colonnes ; WHERE garde les lignes dont la condition est vraie. Une colonne du filtre peut être absente de SELECT. Les constantes et les colonnes comparées doivent avoir des types compatibles. Les commentaires et la mise en forme facilitent la relecture du script.

Atelier 4 : Travail pratique en classe

1. On souhaite prévenir les clients d'une boutique spécifique qu'elle va changer d'adresse. Via la table Sales.Customer, isolez les clients reliés au store id 990.
2. On souhaite vérifier les informations d'un produit spécifique. Via la table Production.Product, isolez les informations du produit ayant pour nom 'Adjustable Race'.
3. On souhaite trouver les produits dont le prix d'achat est égal au prix de vente. Écrivez une requête SQL pour sélectionner les produits de la table production.product dont le standardcost est égal au listprice.
4. On souhaite vérifier que le stock de sécurité est bien toujours supérieur au stock limite de commande. Écrivez une requête SQL pour sélectionner les produits de la table production.product dont le safetystocklevel est inférieur au reorderpoint
5. On souhaite isoler les zones de ventes en dehors des USA. Écrivez une requête SQL pour sélectionner les zones de vente de la table Sales.Salesterritory dont le countryregioncode n'est pas égal à 'US'
6. On souhaite vérifier que tous les produits ont bien un temps de manufacture indiqué. Écrivez une requête SQL pour sélectionner les produits de la table production.product dont le daystomanufacture est vide
7. On souhaite voir si le poids des produits est parfois indiqué. Écrivez une requête SQL pour sélectionner les produits de la table production.product, dont le weight n'est pas vide.

Atelier 4 : Travail pratique à rendre

Mission : fournir un script de consultation filtrée du catalogue AdventureWorks et des personnes. Écrivez les requêtes de manière autonome. Utilisez une seule condition par requête ; adaptez les noms exacts à votre archive. Chaque requête doit être numérotée, commentée et terminée par un point-virgule.

Exercice 1 — Comprendre et corriger

1. Expliquez la différence entre SELECT et WHERE.
2. Corrigez A pour conserver les produits dont la couleur est Red.
3. Corrigez B pour rechercher une couleur absente.
4. Corrigez C pour filtrer sur une date.
5. Expliquez le résultat de la condition listprice >= 500 pour un prix de 500, un prix de 499 et un prix NULL.

```
-- A
SELECT name
FROM production.product;
WHERE color = "Red";

-- B
SELECT name
FROM production.product
WHERE color = NULL;

-- C
SELECT name
FROM production.product
WHERE sellstartdate >= 2013-01-01;
```

Exercice 2 — Filtrer des textes et des nombres

Écrivez et exécutez les requêtes suivantes :

Q1. Afficher identifiant, nom et couleur des produits rouges.

Q2. Afficher seulement numéro et nom des produits rouges, sans afficher la couleur.

Q3. Afficher numéro, nom et prix catalogue des produits dont le prix est supérieur ou égal à 500.

Q4. Afficher les mêmes colonnes pour les prix strictement supérieurs à 500.

Q5. Afficher numéro, nom et couleur des produits dont la couleur renseignée est différente de Red.

Comparez Q1 et Q2 : nombre de colonnes et ensemble de produits retenus. Comparez Q3 et Q4 : les produits à la frontière 500 sont-ils inclus ? Si cette valeur n'existe pas, choisissez un prix existant et documentez l'adaptation. Expliquez pourquoi Q5 n'inclut pas les couleurs NULL.

Exercice 3 — Comparer des colonnes et filtrer les dates

Q6. Afficher numéro, nom, prix catalogue et coût standard des produits dont le prix catalogue est supérieur au coût standard.

Q7. Afficher numéro, nom, stock de sécurité et point de réapprovisionnement des produits dont ces deux niveaux sont égaux.

Q8. Afficher numéro, nom et date de début de commercialisation des produits commercialisés avant le 1er janvier 2014.

Justifiez la différence entre une comparaison avec une constante et une comparaison entre colonnes. Pour Q8, expliquez le littéral DATE et pourquoi une frontière sur un timestamp doit prendre en compte les heures. Un résultat vide est recevable s'il est correctement expliqué.

Exercice 4 — Utiliser LEFT et les tests NULL

Q9. Afficher prénom et nom de famille des personnes dont le nom de famille commence par M.

Q10. Afficher prénom et nom de famille des personnes dont les deux premiers caractères du nom sont Ma. Adaptez le préfixe si nécessaire après observation.

Q11. Afficher numéro et nom des produits dont la date de fin de commercialisation est absente.

Q12. Afficher numéro, nom et couleur des produits dont la couleur est renseignée.

Expliquez pourquoi LEFT(lastname, 2) = 'Ma' est plus restrictif que LEFT(lastname, 1) = 'M'. Distinguez NULL, 0, chaîne vide et texte 'NULL'. Pour Q11, indiquez pourquoi l'absence de date de fin ne suffit pas à prouver qu'un produit est aujourd'hui en vente.